



《AI大模型Ragent项目》——怎么让大模型同时给30个意图节点打分

来自： 拿个offer-开源&项目实战



马丁

2026年04月27日 22:27

开篇引言

上一篇把意图树讲清楚了——三级结构 (DOMAIN → CATEGORY → TOPIC)，三种意图类型 (KB / MCP / SYSTEM)，IntentNode 的字段含义，数据库存扁平、Redis 缓存整棵树。意图树在内存中加载好了，leafNodes 列表已经准备就绪。但准备就绪不等于能直接用。意图树只是一个静态的数据结构，要让它发挥作用，得把叶子节点的信息喂给 LLM，让 LLM 判断用户的问题跟哪个节点最匹配。

带着几个具体的疑问往下看：

叶子节点那么多，怎么把每个节点的信息塞进 Prompt？

Prompt 里怎么告诉 LLM 该按什么规则打分？

LLM 返回的结果是什么格式？万一格式不对怎么办？

Temperature 和 TopP 设多少才合适？

说明：本篇 Prompt 模板中的示例（分类节点、输出示例等）统一用电商智能体场景举例，和 Ragent 代码中的 intent-classifier.st 会有细节差异，但核心规则和结构完全一致。

用一个直观的场景来感受整个过程。假设电商客服的意图树有 11 个叶子节点，用户问了一句：

跨境包裹清关一般要多久？

系统要做的事情是：把 11 个叶子节点全部序列化成本文，拼进 Prompt 模板，一次 LLM 调用搞定所有节点的打分。LLM 返回一个 JSON 数组，里面是每个匹配节点的 ID 和分数。分数最高的那个——清关流程，score=0.92——就是路由目标。

这篇文章要把从叶子节点序列化到 JSON 解析的完整链路走一遍。

叶子节点序列化：LLM 看到的是什

1. buildPrompt() 的序列化逻辑

上一篇提到，只有叶子节点参与打分。DefaultIntentClassifier 的 buildPrompt() 方法负责把这些叶子节点变成 LLM 能读懂的文本。

```
private String buildPrompt(List<IntentNode> leafNodes) {
    StringBuilder sb = new StringBuilder();

    for (IntentNode node : leafNodes) {
        sb.append("- id=").append(node.getId()).append("\n");
        sb.append(" path=").append(node.getFullPath()).append("\n");
        sb.append(" description=").append(node.getDescription()).append("\n");

        // 添加节点类型标识 (V3 Enterprise 支持 MCP)
        if (node.isMCP()) {
            sb.append(" type=MCP\n");
            if (node.getMcpToolId() != null) {
                sb.append(" toolId=").append(node.getMcpToolId()).append("\n");
            }
        } else if (node.isSystem()) {
            sb.append(" type=SYSTEM\n");
        } else {
            sb.append(" type=KB\n");
        }

        if (node.getExamples() != null && !node.getExamples().isEmpty()) {
            sb.append(" examples=");
            sb.append(String.join(" / ", node.getExamples()));
            sb.append("\n");
        }

        sb.append("\n");
    }
}
```

```
    }

    return promptTemplateLoader.render(
        INTENT_CLASSIFIER_PROMPT_PATH,
        Map.of("intent_list", sb.toString())
    );
}
```

逐行拆一下，每个叶子节点被序列化化成五个字段：

- id=xxx：节点的唯一标识。LLM 返回打分结果时，需要原样回传这个 ID，代码端靠它把分数关联回 IntentNode 对象。
- path=xxx：全路径，比如物流与配送 > 跨境物流 > 清关流程。帮助 LLM 理解这个节点在树中的层级关系和所属领域。如果只给 description，LLM 不一定能准确判断它属于哪个业务板块。
- description=xxx：语义说明，是 LLM 匹配的核心依据。描述越清晰，LLM 越容易判断用户问题跟这个节点的相关度。
- type=KB/MCP/SYSTEM：节点类型标识。KB 是知识库检索，MCP 是工具调用，SYSTEM 是系统交互。类型不同，匹配逻辑可能不同——MCP 节点往往需要匹配具体的动作语义（查询、操作），SYSTEM 节点匹配的是社交性问候。
- examples=xxx：示例问题，多个示例用 / 分隔。相当于给 LLM 做 Few-shot——告诉它这类节点通常匹配什么样的问题。

序列化完成后，所有叶子节点的文本拼成一个大字符串，填入 Prompt 模板的 {intent_list} 占位符。

2. 一个完整的分类列表长什么样

用电商客服场景来看，11 个叶子节点序列化后在 Prompt 中的样子：

```
- id=product-info
  path=商品服务 > 商品信息
  description=商品规格、参数、功能介绍、适用场景等产品信息
  type=KB
  examples=iPhone 16 Pro 的摄像头参数是什么？ / 这款耳机支持降噪吗？

- id=after-sales-repair
  path=商品服务 > 售后维修
  description=商品维修政策、保修期限、维修流程、维修网点等信息
  type=KB
  examples=手机屏幕碎了怎么保修？ / 保修期内维修要收费吗？

- id=return-exchange
  path=商品服务 > 退换货政策
  description=退货条件、换货流程、退款时效、七天无理由退货规则等
  type=KB
  examples=买了一周的东西还能退吗？ / 退货运费谁出？

- id=logistics-domestic-delivery
  path=物流与配送 > 国内物流 > 配送方式
  description=国内快递公司选择、配送时效、同城配送、预约配送等说明
  type=KB
  examples=你们用什么快递发货？ / 能选顺丰吗？

- id=logistics-domestic-fee
  path=物流与配送 > 国内物流 > 运费规则
  description=国内物流的运费计算规则、包邮条件、偏远地区加收等
  type=KB
  examples=满多少包邮？ / 新疆发货运费多少？

- id=logistics-overseas-warehouse
  path=物流与配送 > 跨境物流 > 海外仓
  description=海外仓备货、海外仓发货时效、海外仓退货流程等说明
  type=KB
  examples=海外仓发货一般几天到？ / 你们有美国仓吗？

- id=logistics-overseas-customs
  path=物流与配送 > 跨境物流 > 清关流程
```

```
description=跨境物流的清关申报、关税计算、禁运品规则等相关说明
type=KB
examples=海淘包裹清关一般要多久？

- id=logistics-overseas-fee
  path=物流与配送 > 跨境物流 > 运费计算
  description=跨境物流运费计算方式、不同国家和地区的费率对比等
  type=KB
  examples=寄到日本运费怎么算？ / 跨境运费比国内贵多少？

- id=order-query
  path=订单管理 > 订单查询
  description=用户订单状态、物流进度、支付记录等实时查询
  type=MCP
  toolId=order-query-tool
  examples=帮我查一下订单 2024112801 的物流进度 / 我的订单到哪了？

- id=system-greeting
  path=系统交互 > 欢迎与问候
  description=用户打招呼、问好、感谢等社交性问答
  type=SYSTEM
  examples=你好 / 谢谢你

- id=system-about
  path=系统交互 > 关于助手
  description=介绍助手身份、能力范围、使用指南等
  type=SYSTEM
  examples=你是谁？ / 你能做什么？
```

这就是 LLM 在 Prompt 的分类列表部分看到的全部内容。11 个叶子节点，每个四到六行，信息密度不低但结构清晰。

3. 序列化的设计权衡

几个设计选择值得说一下。

为什么用 key=value 格式而不是 JSON？ 可读性好，Token 占用少。如果用 JSON，每个节点要写成 {"id": "logistics-overseas-customs", "path": "物流与配送 > 跨境物流 > 清关流程", ...}——引号、花括号、冒号、逗号全都是额外的 Token。11 个节点下来差距不大，但如果叶子节点有 30 个，省下来的 Token 就可观了。

为什么把 examples 用 / 拼成一行而不是每个示例一行？ 控制长度。11 个节点如果每个有 3 个示例、每个示例独占一行，Prompt 会多出几十行。拼成一行，LLM 照样能理解，但 Prompt 紧凑很多。

为什么要带 type 标识？ 帮助 LLM 理解不同节点的性质。MCP 节点是工具调用——用户的问题通常带有明确的操作意图（查一下、帮我看看）；SYSTEM 节点是系统交互——匹配的是问候、感谢这类社交性问题；KB 节点是知识检索——匹配的是知识性问题。类型信息相当于给 LLM 一个额外的判断维度。

Prompt 模板拆解：每条规则在干什么

叶子节点列表只是 Prompt 的一部分。完整的 Prompt 模板（intent-classifier.st）包含角色定义、判断规则、评分标准、输出规范，加上最后的分类列表。一段一段来看。

1. 角色定义与分类节点说明

```
# 角色定义
企业内部知识库意图分类助手，负责将用户问题路由到正确的知识分类节点。

# 分类节点说明
每个分类节点包含：
- **id**：唯一标识
- **path**：分类树路径（domain / category / topic）
- **description**：知识范围说明
- **examples**：典型问题示例
```

角色定义一句话，告诉 LLM 它的身份和任务边界——你是一个分类助手，不是通用聊天机器人。

分类节点说明是给 LLM 的数据字典。后面分类列表里每个节点都有这四个字段，这里先告诉 LLM 每个字段代表什么含义，读到列表时就不会困惑。

2. 两步判断流程——实体导向 vs 主题导向

这是整个 Prompt 模板中最核心的设计。

```
# 核心判断流程

## 第一步：识别问题类型

| 问题类型 | 特征 | 匹配策略 |
|-----|-----|-----|
| **实体导向** | 包含具体系统/产品/模块/客户名称 | 必须命中关键实体名称（强匹配） |
| **主题导向** | 围绕主题/领域，无具体实体名称 | 匹配 path/description 中的主题词 |

## 第二步：应用匹配规则

### 实体导向问题
- **强匹配要求**：关键实体必须在分类的 id/path/description/examples 中出现
- **不匹配情况**：实体名称不一致时，即使业务领域相似也不匹配
- **无匹配时**：返回空数组 `[]`，不要强行选择相近分类

### 主题导向问题
- **匹配依据**：path/description 中包含相同或高度同义的主题词
- **示例**：
  - "跨境运费怎么算" → 匹配含"运费计算"的分类
  - "海外仓退货流程" → 匹配含"海外仓"的分类

### 相关度优先级
1. 实体/主题名称精确匹配（最高优先）
2. 典型问题场景或主题词匹配
3. 行业/领域相似性（仅作为弱信号，不单独作为选择理由）
```

2.1 为什么要区分两种问题类型

用电商客服的例子来感受。

实体导向问题：用户问“国内顺丰快递能发到新疆吗？”——问题中明确提到了国内和顺丰快递。这类问题的匹配策略是强匹配：国内物流这个方向必须在节点的 id、path、description 或 examples 中出现，才算匹配。

主题导向问题：用户问“运费怎么算？”——问题里只有运费这个主题词，没有指定是国内还是跨境。这类问题用的是弱匹配：只要节点的 path 或 description 中包含运费或语义高度相近的词，就算匹配。

如果不区分这两种类型会怎样？

实体导向的问题用主题匹配策略，可能跨类目匹配到错误的分类。用户明明问的是国内快递，结果因为跨境物流也涉及配送相关话题，LLM 把两个都返回了。

主题导向的问题用实体匹配策略，找不到精确实体就返回空数组，明明有运费规则和运费计算分类完全匹配却漏掉了。

区分实体导向和主题导向，本质是给 LLM 两套不同的匹配策略：强匹配防止跨类目误匹配，弱匹配防止主题类问题漏匹配。

2.2 匹配规则的细节

相关度优先级也有讲究，分三档：

优先级	匹配类型	说明
最高	实体/主题名称精确匹配	问题中的关键词和节点字段中的文本完全一致
中等	典型问题场景或主题词匹配	问题的场景与 examples 中的典型问题相似
最低	行业/领域相似性	仅作为弱信号，不能单独作为选择理由

最低优先级那条很关键。LLM 有时候会因为行业领域相似就给出匹配结果——用户问的是跨境物流，LLM 觉得国内物流和跨境物流都属于物流领域，就把国内物流也返回了。Prompt 里明确告诉它：领域相似性只是弱信号，不能单独作为选择理由。

3. 选择规则——数量控制与类目限定

```
# 选择规则

## 数量控制
- **默认**: 只返回 1 个最核心的主意图分类
- **例外**: 仅当问题明确包含 2 个独立问题且需要不同知识库时, 可返回 2 个 (最多)
- **歧义引导式问答**: 如果问题只包含主题词 (如"运费") 且该主题在多个类目中同名出现, 返回这些同名分类用于引导式问答 (最多 3 个)
- **判断标准**: 已能清楚判断主意图时, 忽略次要意图

## 类目限定规则
- 问题明确提到某类目 (如"跨境物流") 时, 只在该类目分类下选择
- 不要跨类选择 (如问"跨境物流"时不选"国内物流"分类)
- 仅当问题明确比较多个类目时, 才可跨类选择

## 无匹配处理
- 所有分类相关度都较低时, 返回空数组 ``[]``
- 但若问题中出现明确主题词且与分类节点 name/path/description 明确匹配, 不要返回空数组
- 不要为"有结果"而勉强选择弱相关分类
```

3.1 数量控制

默认只返回 1 个, 这是有道理的。上游查询改写 (第 4 篇) 已经把复合问题拆成了子问题, 每个子问题通常只有一个核心意图。一次 classifyTargets() 调用处理的是一个单独的子问题, 不太会出现一个子问题包含两个独立意图的情况。

例外场景有两个。一个是拆分不够干净, 一个子问题里确实包含两个独立问题——最多返回 2 个。另一个是歧义场景: 同名主题出现在多个类目中, 比如用户问“运费怎么算?”, 国内物流和跨境物流下面都有运费相关的分类, 返回最多 3 个候选让下游做引导式问答。歧义引导的细节在第 8 篇展开, 这里知道 Prompt 有这条规则就行。

3.2 类目限定规则

用电商场景打比方: 用户问“退换货的退货政策是什么”, 不应该匹配到物流方向的节点。用户问题里已经明确了退换货这个方向, LLM 不应该跨方向去选。

类目限定规则的存在是防止 LLM 过度联想。LLM 的语义理解能力很强, 有时候强得过头——用户问的是跨境物流的事情, LLM 觉得国内物流也有点关系就顺手返回了。这条规则把这种过度联想压下去。

4. 评分标准——分数区间的含义

```
# 评分标准

| 分数区间 | 匹配程度 | 说明 |
|-----|-----|-----|
| **> 0.8** | 强匹配 | 关键实体/主题名称明确一致, 问题场景高度吻合 |
| **0.4-0.8** | 中等相关 | 部分要素匹配, 但关键实体不完全一致 |
| **< 0.4** | 弱相关 | 仅勉强沾边, 建议返回空数组 ``[]`` |

**整体评估**: 所有候选分类分数均低于 0.6 时, 优先返回空数组 ``[]``
**例外**: 若命中"同名主题多类目"歧义场景, 可返回 0.4-0.7 的多项候选用于引导式问答
```

用之前的例子。用户问“跨境包裹清关一般要多久?”, 清关流程节点的 description 里有跨境物流的清关申报, examples 里有海淘包裹清关一般要多久? ——关键主题完全一致, 场景高度吻合, score 给到 0.92, 属于 > 0.8 的强匹配。

0.4-0.8 是中等相关。比如用户问“跨境包裹要交多少税?”, 清关流程的 description 有关税计算, 但问题问的是具体金额而不是流程, 匹配不完全——可能给个 0.65。

< 0.4 是弱相关, 建议返回空数组。还有一条整体评估规则: 所有候选分数都低于 0.6 时, 优先返回空数组。

这里有一个重要的设计意图: 宁可返回空数组让系统走兜底逻辑 (全局搜索或引导澄清), 也不要勉强匹配到弱相关的节点导致检索方向错误。错误的路由比没有路由更糟糕——路由错了, 从错误的知识库里检索到不相关的 chunk, LLM 基于这些 chunk 生成的答案大概率也是错的, 而且用户看得出来答案是从错误的方向来的。

5. 输出规范——JSON 格式约束

```
# 输出规范

## 格式要求
```

```
- 只输出 JSON 数组，无其他文字
- 数组元素字段：
  - `id`：字符串，对应分类列表中的 id（严格一致）
  - `score`：数值，范围 0~1
  - `reason`：字符串，说明为什么该分类最合适（不要复述问题）

## 输出示例
[
  {"id": "logistics-overseas-customs", "score": 0.92, "reason": "问题询问跨境清关时效，直接对应清关流程分"},
  {"id": "logistics-overseas-fee", "score": 0.75, "reason": "问题涉及跨境物流费用，与运费计算分类部分相关"}
]

## 歧义引导式问答示例
[
  {"id": "logistics-domestic-fee", "score": 0.62, "reason": "问题仅包含\"运费\"主题，国内物流下有运费规则分"},
  {"id": "logistics-overseas-fee", "score": 0.60, "reason": "问题仅包含\"运费\"主题，跨境物流下有运费计算分"}
]

## 空结果示例
[]
```

三个字段各有用处：

- id：严格对应分类列表中的 id。代码端靠这个 id 从 id2Node 查找表中关联回 IntentNode 对象。
- score：0~1 的浮点数，下游用它做过滤和排序。
- reason：匹配理由。代码中没有使用 reason 字段做逻辑判断，但它对调试非常有价值——在日志中能看到 LLM 为什么选了这个节点，帮你判断 Prompt 规则是不是在起作用。

Prompt 给了三种输出示例：正常匹配、歧义引导式问答、空结果。三个示例覆盖了 LLM 可能遇到的主要输出场景，减少 LLM 在格式上犯错的概率。

最后是分类列表，用 {intent_list} 占位符标记：

```
# 分类列表
{intent_list}
```

运行时，buildPrompt() 序列化出来的叶子节点文本会填到这里。

LLM 调用：低温度保确定性

1. 消息结构

classifyTargets() 构建的 ChatRequest 只有两条消息：

```
ChatRequest request = ChatRequest.builder()
    .messages(List.of(
        ChatMessage.system(systemPrompt),
        ChatMessage.user(question)
    ))
    .temperature(0.1D)
    .topP(0.3D)
    .thinking(false)
    .build();
```

System 消息：完整的 Prompt 模板——角色定义 + 判断规则 + 评分标准 + 输出规范 + 分类列表。所有规则和数据都在这一条消息里。

User 消息：用户的问题，就一句话。

为什么这么简洁？意图分类是单轮任务，不需要多轮对话历史。System Prompt 已经包含了 LLM 做判断需要的全部信息——规则在模板里，数据在分类列表里，输入就是用户问题。两条消息，一轮调用，干干净净。

2. Temperature 和 TopP 的选择

Temperature 和 TopP 这两个参数直接决定 LLM 输出的随机性。

参数组合	特点	适用场景
Temperature=0.1, TopP=0.3	极低随机性，输出高度确定	意图分类、结构化输出
Temperature=0.7, TopP=1.0	适中随机性，有一定创造性	通用对话、内容生成
Temperature=1.0+, TopP=1.0	高随机性，输出多样	创意写作、头脑风暴

Ragent 选择 Temperature=0.1 + TopP=0.3，理由很直接——意图分类是分析性任务，需要的是确定性，不是创造性。想象一下如果 Temperature 设成 0.7 会怎样。用户问“跨境包裹清关一般要多久？”，今天 LLM 返回清关流程 score=0.92，明天同样的问题可能返回运费计算 score=0.85。用户体验上就是：同一个问题今天能答对，明天就答偏了。路由不稳定，整个系统的表现就不可预测。TopP=0.3 进一步收窄候选 Token 的范围，减少输出的随机波动。Temperature 控制的是概率分布的平坦程度，TopP 控制的是采样范围的大小，两个参数叠加使用效果更好。

有人可能问：为什么不用 Temperature=0？理论上 Temperature=0 最确定，但部分模型在 Temperature=0 时行为不够稳定（存在数值精度问题），0.1 是工程上常用的安全值。

3. thinking=false：不需要深度思考

thinking(false) 关闭深度思考模式。意图分类是模式匹配任务——把用户问题和节点描述做语义对比，不需要 Chain of Thought（思维链）推理。开启深度思考会增加 Token 消耗和响应延迟，但对分类准确率的提升微乎其微。省掉这部分开销，意图分类环节的延迟和成本都能压下来。

JSON 解析的鲁棒容错

1. 为什么需要容错

Prompt 里已经写了只输出 JSON 数组，无其他文字，但 LLM 的输出不是 100% 可控的。实际跑起来会遇到各种意外情况：有些模型自动给 JSON 加 Markdown 代码块标记，有些模型喜欢在数组外面包一层对象，有些元素的字段可能缺失，LLM 偶尔还会编造不存在的 ID。Prompt 能约束大方向，但不能保证每次输出都完美符合格式。所以代码端必须有足够的容错能力。

2. 五层容错机制

classifyTargets() 的 JSON 解析部分设计了五层容错，逐层递进：

2.1 Markdown 代码块清除

```
String cleanedRaw = LLMResponseCleaner.stripMarkdownCodeFence(raw);
```

LLMResponseCleaner 的实现：

```
public final class LLMResponseCleaner {

    private static final Pattern LEADING_CODE_FENCE = Pattern.compile("^```[\\w-]*\\s*\\n?");
    private static final Pattern TRAILING_CODE_FENCE = Pattern.compile("\\n?```\\s*$");

    public static String stripMarkdownCodeFence(String raw) {
        if (raw == null) return null;
        String cleaned = raw.trim();
        cleaned = LEADING_CODE_FENCE.matcher(cleaned).replaceFirst("");
        cleaned = TRAILING_CODE_FENCE.matcher(cleaned).replaceFirst("");
        return cleaned.trim();
    }
}
```

有些模型会自动给 JSON 输出加上 ```json 和 ``` 围栏。对人类来说这是友好的格式化，对代码来说就是多了两行需要剥掉的噪音。两个正则，一个去头一个去尾，干净利落。

2.2 两种 JSON 格式兼容

```
JsonElement root = JsonParser.parseString(cleanedRaw);

JsonArray arr;
if (root.isJsonArray()) {
    arr = root.getAsJsonArray();
} else if (root.isJsonObject() && root.getAsJsonObject().has("results")) {
    // 容错：如果模型外面又包了一层 { "results": [...] }
    arr = root.getAsJsonObject().getAsJsonArray("results");
} else {
    log.warn("LLM 返回了非预期的 JSON 格式，原始响应：{}", raw);
    return List.of();
}
```

正常情况 LLM 直接返回 [...] 数组。但有些模型倾向于在外面包一层对象：{"results": [...]}。代码兼容这两种格式——是数组就直接用，是对象就取 results 字段。如果两种都不是，打 warn 日志返回空列表。

2.3 缺失字段跳过

```
for (JsonElement el : arr) {
    if (!el.isJsonObject()) continue;
    JsonObject obj = el.getAsJsonObject();

    if (!obj.has("id") || !obj.has("score")) continue;
    // ...
}
```

遍历数组时，如果某个元素不是 JSON 对象，跳过；如果缺少 id 或 score 字段，也跳过。不因为一个坏元素丢掉整批结果。假设 LLM 返回了 3 个元素，其中 1 个格式有问题，另外 2 个没问题——那就用那 2 个，而不是整个解析失败。

2.4 未知 ID 校验

```
IntentNode node = data.id2Node.get(id);
if (node == null) {
    log.warn("LLM 返回了未知的意图节点 ID: {}, 已跳过", id);
    continue;
}
```

LLM 返回的 id 在 id2Node 查找表中找不到。可能的原因：LLM 编造了一个不存在的 ID，或者拼写有偏差。跳过并打 warn 日志——日志里记录了 LLM 编造的 ID，方便排查 Prompt 是不是需要调整。

2.5 全局 try-catch 兜底

```
try {
    // 整个解析逻辑
    // ...
    return scores;
} catch (Exception e) {
    log.warn("解析 LLM 响应失败，原始内容：{}", raw, e);
    return List.of();
}
```

整个解析逻辑包在 try-catch 里。如果 JSON 完全无法解析（比如 LLM 输出了自然语言而不是 JSON），兜底返回空列表。空列表到下游会触发全局搜索的兜底逻辑，不会让整个流程中断。

五层容错用一句话概括：**能解析多少就解析多少，解析不了就优雅降级。**

3. 排序、日志副作用与下游消费

解析完成后，按 score 降序排序，封装成 List<NodeScore> 返回：

```
scores.sort(Comparator.comparingDouble(NodeScore::getScore).reversed());
```

NodeScore 就两个字段，结构很简单：

```
@Data
@AllArgsConstructor
@Builder
public class NodeScore {
    private IntentNode node; // 意图节点
    private double score;    // LLM 打分结果 (0~1)
}
```

在 return 之前有一段调试日志，这里藏了一个值得注意的坑：

```
log.info("当前问题: {} \n意图识别树如下所示: {} \n",
        question,
        JSONUtil.toJsonPrettyStr(
            scores.stream().peek(each -> {
                IntentNode node = each.getNode();
                node.setChildren(null); // ⚠️ 副作用
            }).collect(Collectors.toList())
        )
    );
```

为了让日志输出不被嵌套的 children 撑爆（叶子节点的 children 是空列表，但 IntentNode 对象上还挂着从缓存里读出来的引用），代码用 peek() 里的 setChildren(null) 把节点的 children 置空了。

问题在于：这修改了共享的 IntentNode 对象。peek() 是一个中间操作，它对流中的每个元素执行操作时修改的是原始对象，不是副本。当前流程中后续不再依赖 children（打分只关心叶子节点），所以实际不影响业务。但如果将来有人在 classifyTargets() 返回之后还需要用到 children，就会踩一个很难排查的空指针。

这是实际项目中典型的为了调试日志引入副作用的坑。更安全的做法是序列化时用 @JsonIgnore 或自定义序列化器来排除 children，而不是修改原始对象。

接下来看下游怎么消费打分结果。IntentResolver 做两件事：

```
// 常量（定义在 RAGConstant 类中，IntentResolver 通过静态导入引用）
public static final double INTENT_MIN_SCORE = 0.35;
public static final int MAX_INTENT_COUNT = 3;

// 单个问题的过滤逻辑
private List<NodeScore> classifyIntents(String question) {
    List<NodeScore> scores = intentClassifier.classifyTargets(question);
    return scores.stream()
        .filter(ns -> ns.getScore() >= INTENT_MIN_SCORE)
        .limit(MAX_INTENT_COUNT)
        .toList();
}
```

过滤掉分数低于 INTENT_MIN_SCORE (0.35) 的节点。

限制每个问题最多保留 MAX_INTENT_COUNT (3) 个意图。

你可能注意到了：Prompt 里说 < 0.4 建议返回空数组，但代码端的最低分数线是 0.35。为什么不一样？Prompt 里的 0.4 是给 LLM 的建议，LLM 不一定严格遵守。有些边界情况 LLM 可能给出 0.38 的分数，代码端用 0.35 做兜底，留一点容差空间。

多个子问题的分类在 IntentResolver.resolve() 中通过 CompletableFuture 并行执行：

```
@RagTraceNode(name = "intent-resolve", type = "INTENT")
public List<SubQuestionIntent> resolve(RewriteResult rewriteResult) {
    List<String> subQuestions = CollUtil.isEmpty(rewriteResult.subQuestions())
        ? rewriteResult.subQuestions()
```

```
        : List.of(rewriteResult.rewrittenQuestion());

// 每个子问题通过 CompletableFuture 并行分类, 互不影响
List<CompletableFuture<SubQuestionIntent>> tasks = subQuestions.stream()
    .map(q -> CompletableFuture.supplyAsync(
        () -> {
            try {
                return new SubQuestionIntent(q, classifyIntents(q));
            } catch (Exception e) {
                log.error("子问题意图分类失败, 降级为空意图, question: {}", q, e);
                return new SubQuestionIntent(q, List.of()); // 单个子问题失败不影响其他
            }
        },
        intentClassifyExecutor // 专用线程池
    ))
    .toList();

List<SubQuestionIntent> subIntents = tasks.stream()
    .map(CompletableFuture::join)
    .toList();

return capTotalIntents(subIntents); // 封顶算法: 第 7 篇详细展开
}
```

几个关键点:

每个子问题独立调一次 LLM, 通过 `CompletableFuture.supplyAsync()` 提交到 `intentClassifyExecutor` 专用线程池并行执行。三个子问题不用串行等, 并行跑完时间只取最慢的那一个。

单个子问题的分类失败降级为空意图列表 (`List.of()`), 不会阻断其他子问题——这是隔离性设计。一个子问题的 LLM 调用超时或者解析失败, 其他子问题的结果不受影响。

所有子问题分类完成后, `capTotalIntents()` 做全局封顶。如果总意图数超过 `MAX_INTENT_COUNT` (3), 每个子问题至少保留 1 个最高分意图, 剩余配额按分数从高到低分配。封顶算法的具体策略在第 7 篇展开。

Prompt 模板的加载与渲染

顺带讲一下 Prompt 模板是怎么从文件变成最终发给 LLM 的文本的。

模板文件 `intent-classifier.st` 放在 `src/main/resources/prompt/` 目录下, 通过 `PromptTemplateLoader` 加载:

```
@Service
public class PromptTemplateLoader {

    private final ResourceLoader resourceLoader;
    private final Map<String, String> cache = new ConcurrentHashMap<>();

    public String load(String path) {
        return cache.computeIfAbsent(path, this::readResource);
    }

    public String render(String path, Map<String, String> slots) {
        String template = load(path);
        String filled = PromptTemplateUtils.fillSlots(template, slots);
        return PromptTemplateUtils.cleanupPrompt(filled);
    }
}
```

`PromptTemplateUtils` 的实现也很直白:

```
public final class PromptTemplateUtils {
    private static final Pattern MULTI_BLANK_LINES = Pattern.compile("(\\n){3,}");
```

```
public static String fillSlots(String template, Map<String, String> slots) {
    String result = template;
    for (Map.Entry<String, String> entry : slots.entrySet()) {
        result = result.replace("{key} + entry.getKey() + "}", StrUtil.emptyIfNull(entry.getValue
    }
    return result;
}

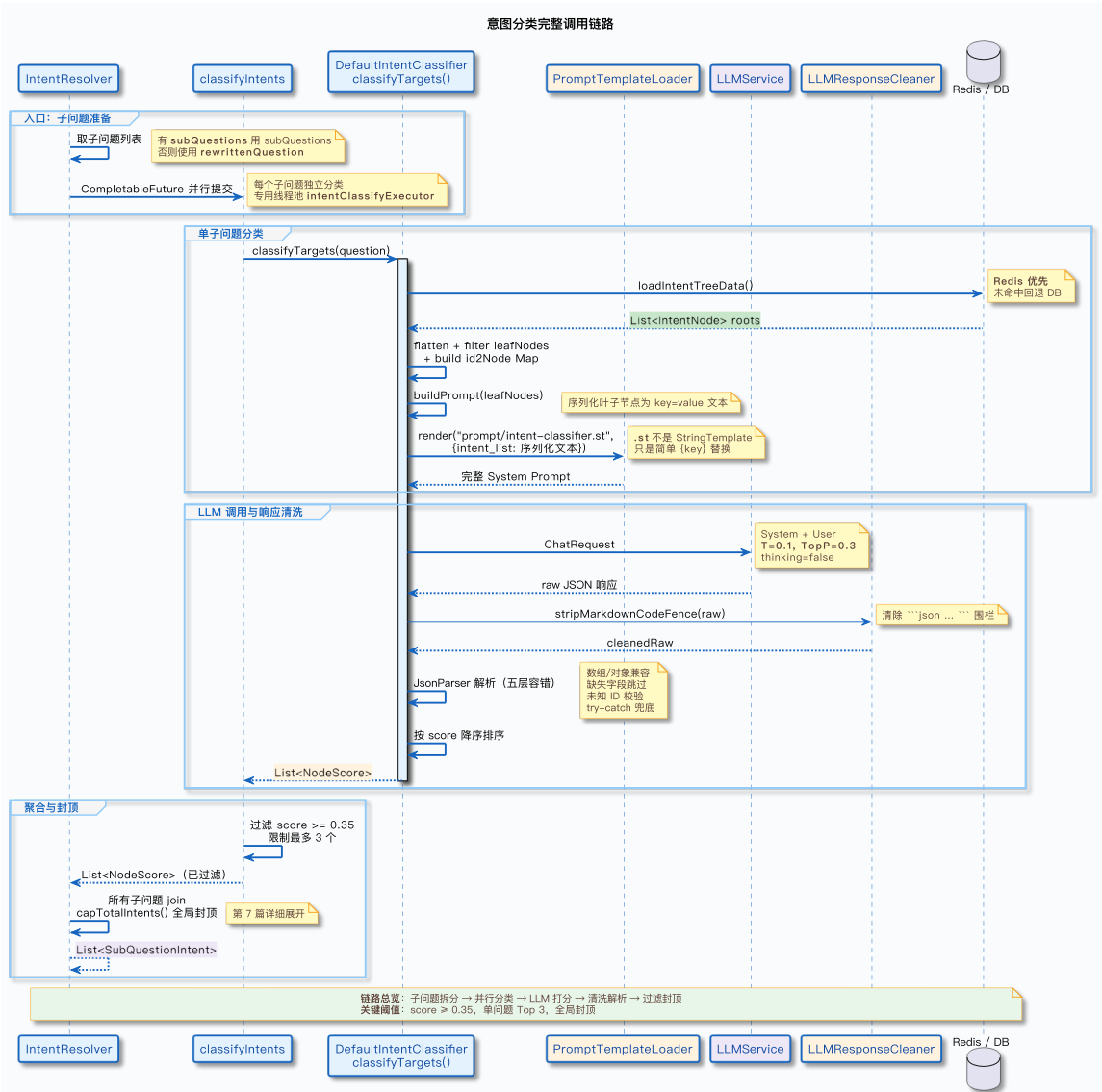
public static String cleanupPrompt(String prompt) {
    return MULTI_BLANK_LINES.matcher(prompt).replaceAll("\n\n").trim();
}
}
```

有一个细节要澄清：虽然模板文件后缀是 .st，但 Ragent 并没有使用 StringTemplate 引擎。fillSlots() 做的就是简单的 String.replace("{key}", value) 替换——模板里只有一个占位符 {intent_list}，加载后用 fillSlots 替换成序列化的叶子节点列表。不引入模板引擎依赖，实现简单，也不容易出问题。

ConcurrentHashMap 缓存模板内容，避免每次都从 classpath 读文件。cleanupPrompt 清理连续 3 个以上空行为 2 个，保证 Prompt 格式整洁——叶子节点之间可能有多余的空行，清理后看起来干净，Token 浪费也少一些。

完整调用链路

把前面讲的所有环节串起来，用一张时序图看整个数据流：



用文字再梳理一遍关键步骤：

- 1. IntentResolver.resolve() 拿到子问题列表，通过 CompletableFuture 并行提交到专用线程池。
- 2. 每个子问题调用 classifyIntents()，内部调用 DefaultIntentClassifier.classifyTargets()。

3. `classifyTargets()` 从 Redis / DB 加载意图树，提取叶子节点。
4. `buildPrompt()` 把叶子节点序列化为 `key=value` 文本，通过 `PromptTemplateLoader` 填入模板的 `{intent_list}` 占位符。
5. 构建 `ChatRequest` (System + User 两条消息, `Temperature=0.1`, `TopP=0.3`, `thinking=false`)，调用 LLM。
6. LLM 返回后，`LLMResponseCleaner` 清理 Markdown 代码块。
7. `JsonParser` 解析 JSON，五层容错保证鲁棒性。
8. 解析结果封装为 `NodeScore`，按 `score` 降序排序返回。
9. `classifyIntents()` 过滤 `score >= 0.35` 并限制最多 3 个。
10. 所有子问题完成后，`capTotalIntents()` 做全局封顶。

小结

回顾本篇的核心要点：

1. 叶子节点序列化为 `id / path / description / type / examples` 的 `key=value` 格式，填入 `intent-classifier.st` 模板的 `{intent_list}` 占位符。模板文件虽然后缀是 `.st`，但不是 `StringTemplate` 引擎，就是简单的 `{key}` 占位符替换。
2. Prompt 模板的核心设计是两步判断流程——先识别问题类型（实体导向 vs 主题导向），再应用对应的匹配规则。实体导向要求强匹配（名称必须出现），主题导向可以弱匹配（主题词语义相近即可）。
3. 数量控制默认只返回 1 个，配合上游的子问题拆分，每个子问题独立做意图分类。歧义场景可返回最多 3 个候选用于引导式问答（第 8 篇展开）。
4. 评分标准分三档（> 0.8 强匹配 / 0.4~0.8 中等 / < 0.4 弱相关），宁可返回空数组也不勉强匹配。
5. `Temperature=0.1` + `TopP=0.3` 保证意图分类的确定性和稳定性，`thinking=false` 省掉不必要的深度思考开销。
6. JSON 解析有五层容错——代码块清除、格式兼容、缺失字段跳过、未知 ID 校验、全局 `try-catch` 兜底。
7. 多个子问题通过 `CompletableFuture` 并行分类，单个失败降级为空意图不阻断其他，下游 `capTotalIntents()` 做全局封顶。

到这里，一个子问题调一次 LLM 的流程已经讲完了。但实际场景中，查询改写可能拆出三个子问题，三个子问题就是三次 LLM 调用。每次可能返回 1~3 个意图，三个子问题加起来可能命中了八个意图。系统总共只允许保留 3 个——该保留哪几个？怎么保证每个子问题至少有一个意图被保留，不至于某个子问题的答案完全没着落？下一篇讲封顶算法——三个子问题命中了八个意图，该保留哪几个。